■ Presuntinho V4.1 — Orchestrated Slash Goal

Mobile-first PWA study hub · SvelteKit + Vite + Dexie · 3-agent orchestration (Skander 1 / Skander 2 / Piccolo). Copy-paste this into a new session. Live site: ***https://presuntinho.netlify.app/***

```
/goal I'm Daniel/Skander, working with Fatma. We're rebuilding **Presuntinho** from a single static HTML page into a real **personal study & life hub PWA**. Site is LIVE at https://presuntinho.netlify.app/ — you must not break it during the refactor; Netlify auto-deploys from `main`.

**You are the principal orchestrator.** Do NOT implement directly. Delegate to **Skander 1** (reasoning/review), **Skander 2** (mechanical implementation), and **Piccolo** (tiny tasks) via `delegate_task`. See "Orchestration model" in EXECUTION PLAN below.

> **v4.1 — Orchestrated plan (2026-06-27).** Replaces v4.0's "you do everything" plan with a 3-agent orchestration model for speed + quality.

# MISSION

Transform the existing repo at `C:\Users\rafaa\Documents\GitHub\presuntinho` (currently HTML+JS+CSS vanilla) into a properly architected **SvelteKit + Vite PWA** with a **plugin-style sub-app registry** that can host many future "apps" (school, work, finances, habits, library, etc.). Mobile-first, but must look and feel premium on PC, TV, tablet, and phone. Add password-gated login.

# CRITICAL — DO NOT LOSE OR BREAK ANY OF THIS

The current site has real content, real interactivity, and real deliverables. **Every single one of these must be preserved and work after the rebuild.** Before touching anything, read each file in the repo and enumerate what you will preserve. Then commit a `PRESERVATION.md` checklist to the repo root, and check off items as you rebuild.

## Content / files (must remain reachable in the new app)

1. **All 5 audio files** in `/assets/` (move to `/static/audio/` in SvelteKit):
- `audio_intro_en.mp3` (1361 KB) — English walkthrough
- `audio_intro_pt-PT.mp3` (351 KB) — Portuguese walkthrough
- `intro_swot.mp3` (664 KB)
- `persona_problem.mp3` (858 KB)
- `tows_recommendation.mp3` (782 KB)
- `/assets/buyer-persona-template-v3.png` (the persona visual)
2. **Documents in `/docs/`** — keep, link from a "Downloads" page:
- `Equivalenza_Mid_Term_Fatma.pdf` (161 KB)
- `Equivalenza_Mid_Term_Fatma.docx` (2248 KB)
3. **The ZIP `equivalenza-midterm-deliverables-V3.zip`** — keep, link from Downloads.
4. **All 9 HTML pages** that currently exist (Home, The Case, Course, Walkthrough, Secrets, Quizzes, Writing, PT, Downloads). Each becomes a route in SvelteKit. **Do not merge pages.** Each gets its own URL.

## Interactivity / state (must keep working)

5. **Easter eggs — every single one**:
- ❤■ **Heart button** (`heartClick` in `assets/js/easter-eggs.js:42`) — 5 tiers, XP rewards, speed bonus, confetti, mascot messages
- ■ **Logo click** (`logoClick`) — 3-click secret, mascot easter egg (`mascotClick`), unlock bonus
- ■ **Konami code** — `↑↑↓↓←→←→BA`, unlocks secret + badge
- ■■ **Keyword detector** (`keyBuf`) — typing `perfume`, `behi`, `fatma` triggers effects
- ■ **Footer click** (`footerClick`) — counter easter egg
- ■ **Secret Room modal** (`closeSRoom`)
- 8+ **secret definitions** in `SECRET_DEFS` array — each unlockable via different trigger
6. **State** (currently in `state.js`, schema in `state_keys`): `xp`, `badges` (8 badge IDs: b7, b8, b9, b10, b12, b13, b14, b15), `visited`, `heartClicks`, `logoClicks`, `logoTimer`, `konamiProg`, `keyBuf`, `footerClicks`, `quizScore`, `quizAnswered`, `secretDiscovered`. **Persist across reloads.** Migrate from localStorage → Dexie/IndexedDB (same data shape, no breaking change for users).
7. **Confetti** (`fireConfetti` in `state.js`) — keep animation identical.
8. **Toast notifications** (`showToast`) — keep.
9. **Navigation** (`navGo`, `navLink`) — keep; refactor to SvelteKit router but expose same function names where possible.
10. **Module Progress bar** (`updateProgress`, `pg`, `pct`) — Read/Quizzes/Writing percentage cards. **Must still render on home.**
11. **Badge grid** (`renderBadges`, `BADGES`, `awardBadge`) — 10+ badge cards with unlock animation.
12. **4 quizzes** (q1, q2, q3, q4) plus Portuguese quiz (`ptq` / `pt-quiz`) — each is its own page, NOT one page with all 5.
13. **Portuguese section** — keep `■■ PT` tab with the Portuguese quiz and audio walkthrough.

## Constraints from previous work (must respect)
```

- **NO build step that breaks Netlify deploy.** Netlify must still auto-deploy `main`. Verify `netlify.toml` builds with the new toolchain before pushing.
- **Netlify config must include** cache headers + security headers (already there) — keep them.
- **HTTPS only**, `strict-origin-when-cross-origin`, `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff` — all must persist in `[[headers]]`.
- **`<meta name="viewport" content="width=device-width, initial-scale=1.0, viewport-fit=cover">`** — must be present for mobile.

REQUIRED NEW FEATURES

1. Password-gated login

- **Splash screen first** (before any content). Two password fields:
- Primary: `I have the best boyfriend in the world`
- Secret/easter-egg shorter: `Fofinho`
- Both unlock the app. Both stored as **scrypt-hashed** in `/static/auth/hashes.json` (use Web Crypto API at build time, or commit pre-hashed values).
- Wrong password → shake animation + try again. 3 wrong attempts → 30s lockout (local).
- After unlock: persist session in `sessionStorage` (auto-lock on tab close) **or** give a "Remember me" toggle that uses localStorage.
- Passwords are **case-sensitive** with leading/trailing whitespace trimmed.
- Display a small ❤■ "Made for Fatma" credit on the splash.

2. Hub / dashboard with sub-app registry

The home page becomes a **central hub**. Layout:

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ ■ Presuntinho Hub [XP] [Badges] [■] ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ ■■■■■■■■■■■ ■■■■■■■■■■■ ■■■■■■■■■■■ ■
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■
■ ■ Escola ■ ■Trabalhos■ ■Finanças■ ■
■ ■■■■■■■■■■■ ■■■■■■■■■■■ ■■■■■■■■■■■ ■
■ ■■■■■■■■■■■ ■■■■■■■■■■■ ■■■■■■■■■■■ ■
■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■
■ ■Hábitos ■ ■Bibliot.■ ■ Defs ■ ■
■ ■■■■■■■■■■■ ■■■■■■■■■■■ ■■■■■■■■■■■ ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Recent activity / streak / motivation ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

Plugin registry (`src/lib/registry.ts`):
```ts
export interface SubApp {
id: string;
name: string;
icon: string;
color: string;
description: string;
route: string; // /escola, /trabalhos, etc
enabled: boolean;
order: number;
}
export const subApps: SubApp[] = [
{ id: 'escola', name: 'Escola', icon: '■', ... },
{ id: 'trabalhos', name: 'Trabalhos', icon: '■', ... },
{ id: 'financas', name: 'Finanças', icon: '■', ... },
{ id: 'habitos', name: 'Hábitos', icon: '■', ... },
{ id: 'biblioteca', name: 'Biblioteca', icon: '■', ... },
];
```

To add a future sub-app: create folder, add one entry to this array. Dashboard re-renders automatically.

3. Escola sub-app (courses / lessons)

Routes:
- `/escola` — list of courses (currently 2: Equivalenza course, Portuguese course)
- `/escola/curso/[slug]` — single course, list of lessons
- `/escola/licao/[courseSlug]/[lessonSlug]` — single lesson page (text + audio + key points + "next" button)
- `/escola/quiz/[quizSlug]` — single quiz page (one quiz per page, not 5 on one page)

Lesson data structure (`src/lib/data/lessons/equivalenza/swot.json`):
```json
{
"id": "swot",
"courseId": "equivalenza",
"title": "SWOT Analysis",
"order": 1,
"audio": "/audio/intro_swot.mp3",
"duration": "~6 min",
"sections": [
{ "type": "h2", "text": "Strengths" },
{ "type": "p", "text": "..." },
{ "type": "ul", "items": ["...", "..."] },
{ "type": "callout", "variant": "insight", "text": "..." }
],
"quiz": "q1",
"keyPoints": ["...", "..."]
}
```

Extract the existing course content (SWOT, persona, problem, TOWS, recommendation) into **structured lesson JSONs**. Each becomes its own page. Currently the content is all on one big page — split it.

4. Trabalhos sub-app (assignments)

Routes:
- `/trabalhos` — list of assignments
- `/trabalhos/assignment/[slug]` — single assignment hub (deadline, deliverables, downloads)

Currently 1: Equivalenza mid-term. Move it here. Keep all docs/ + ZIP download links inside.

5. Finanças sub-app (MVP for v4 — small but real)

Routes:
- `/financas` — dashboard (total balance, monthly spend chart, top categories)
- `/financas/transacoes` — list of transactions with filters
- `/financas/orcamento` — monthly budget tracker
- `/financas/nova` — add transaction form

Data model (Dexie):
```ts
db.transacoes: { id, date, amount, category, description, type: 'income'|'expense' }
db.orcamentos: { id, month: 'YYYY-MM', category, limit, spent }
db.categorias: { id, name, icon, color }
```

Categories seed: Alimentação, Transporte, Habitação, Saúde, Educação, Lazer, Roupa, Beleza, Presentes, Outros.

6. Hábitos sub-app (MVP)

Routes:
- `/habitos` — list of habits with current streak
- `/habitos/habit/[slug]` — habit detail, calendar heatmap, log button
- `/habitos/novo` — add habit

Data model: `habitos` table + `habit_logs` (date, habitId, done).

7. Biblioteca sub-app (MVP)

Just a bookmark manager for now: title, url, tags, description. Routes: `/biblioteca`, `/biblioteca/novo`, `/biblioteca/item/[id]`.

8. Settings (`/definicoes`)

- Reset password
- Clear local data (with confirmation modal)
- Theme: light / dark / auto
- Language: pt-PT / en (i18n via svelte-i18n)
- Export all data as JSON
- Import JSON backup

ARCHITECTURE DECISIONS (validate, don't blindly trust)

Stack

- **Framework: SvelteKit 2 + Svelte 5** (runes). Why: smallest bundle (~28 KB gz), best mobile perf, file-based routing matches the multi-sub-app architecture.
- **Build: Vite 5/6** (default with SvelteKit).
- **Storage: Dexie.js 4** (IndexedDB wrapper). Replace current localStorage. Migration must preserve existing user state (read old `localStorage` key on first run, copy to Dexie, then keep both during transition).
- **PWA: `@vite-pwa/sveltekit`** — auto-generates manifest.json + service worker. **Add to home screen** prompt. Offline-first.
- **Auth: client-side only**. Web Crypto API scrypt. Hash both passwords at build time (Node script), commit JSON. Client compares hash, never stores plaintext.
- **CSS: Svelte scoped styles + global CSS variables for design tokens** (colors, spacing, type scale, shadows). No Tailwind for v4 (overkill, slower cold start on mobile). Revisit later if needed.
- **TypeScript** throughout. Strict mode.
- **Charts (finanças):** lightweight — `chart.js` or pure SVG. No heavy lib.
- **Date handling:** `date-fns` tree-shaken.
- **Icons:** emoji + selective `lucide-svelte` for UI controls (settings cog, close x, etc).
- **i18n:** `svelte-i18n`. PT-PT primary (Fatma is Portuguese), EN secondary. All UI strings externalized from day one.
- **No backend** for v4. All data lives client-side (Dexie). Note this as a future v5 decision.

Folder structure

```
presuntinho/
■■■ src/
■ ■■■ app.html
■ ■■■ app.css # design tokens, resets
■ ■■■ lib/
■ ■ ■■■ registry.ts # sub-app plugin registry
■ ■ ■■■ components/
■ ■ ■ ■■■ HubCard.svelte
■ ■ ■ ■■■ LessonRunner.svelte
■ ■ ■ ■■■ QuizRunner.svelte
■ ■ ■ ■■■ ProgressBar.svelte
■ ■ ■ ■■■ Badge.svelte
■ ■ ■ ■■■ Toast.svelte
■ ■ ■ ■■■ Confetti.svelte
■ ■ ■ ■■■ SecretModal.svelte
■ ■ ■ ■■■ ...
■ ■ ■■■ state/
■ ■ ■ ■■■ db.ts # Dexie schema
■ ■ ■ ■■■ stores/
■ ■ ■ ■ ■■■ user.ts
■ ■ ■ ■ ■■■ progress.ts
■ ■ ■ ■ ■■■ easterEggs.ts
■ ■ ■ ■ ■■■ settings.ts
■ ■ ■ ■■■ migration.ts # localStorage → Dexie
■ ■ ■■■ data/
■ ■ ■ ■■■ lessons/
■ ■ ■ ■ ■■■ equivalenza/
■ ■ ■ ■ ■ ■■■ swot.json
■ ■ ■ ■ ■ ■■■ persona.json
■ ■ ■ ■ ■ ■■■ problem.json
■ ■ ■ ■ ■ ■■■ tows.json
■ ■ ■ ■ ■ ■■■ recommendation.json
■ ■ ■ ■ ■■■ portugues/
■ ■ ■ ■ ■■■ ...
■ ■ ■ ■■■ quizzes/
■ ■ ■ ■ ■■■ q1.json
■ ■ ■ ■ ■■■ q2.json
■ ■ ■ ■ ■■■ q3.json
■ ■ ■ ■ ■■■ q4.json
■ ■ ■ ■ ■■■ pt.json
■ ■ ■ ■■■ secrets.json # 8+ secret definitions
■ ■ ■ ■■■ easter-eggs.json # konami, keywords, triggers
■ ■ ■■■ auth/
■ ■ ■ ■■■ hash.ts # scrypt
■ ■ ■ ■■■ session.ts # sessionStorage mgmt
■ ■ ■ ■■■ splash.ts
■ ■ ■■■ i18n/
■ ■ ■■■ index.ts
■ ■ ■■■ pt-PT.json
■ ■ ■■■ en.json
```

```
■ ■■■ routes/
■ ■■■ +layout.svelte # nav, login gate, theme
■ ■■■ +layout.ts # load session, init db
■ ■■■ +page.svelte # hub dashboard
■ ■■■ splash/
■ ■ ■■■ +page.svelte # password gate (NO nav here)
■ ■■■ escola/
■ ■ ■■■ +page.svelte
■ ■ ■■■ curso/[slug]/+page.svelte
■ ■ ■■■ licao/[courseSlug]/[lessonSlug]/+page.svelte
■ ■ ■■■ quiz/[quizSlug]/+page.svelte
■ ■■■ trabalhos/
■ ■ ■■■ +page.svelte
■ ■ ■■■ assignment/[slug]/+page.svelte
■ ■■■ financas/
■ ■ ■■■ +page.svelte
■ ■ ■■■ transacoes/+page.svelte
■ ■ ■■■ orcamento/+page.svelte
■ ■ ■■■ nova/+page.svelte
■ ■■■ habitos/
■ ■ ■■■ +page.svelte
■ ■ ■■■ habit/[slug]/+page.svelte
■ ■ ■■■ novo/+page.svelte
■ ■■■ biblioteca/
■ ■ ■■■ +page.svelte
■ ■ ■■■ novo/+page.svelte
■ ■ ■■■ item/[id]/+page.svelte
■ ■■■ definicoes/+page.svelte
■ ■■■ (legacy)/ # preserve existing 9 pages temporarily
■ ■■■ case/+page.svelte
■ ■■■ course/+page.svelte
■ ■■■ walk/+page.svelte
■ ■■■ secrets/+page.svelte
■ ■■■ quiz/+page.svelte
■ ■■■ write/+page.svelte
■ ■■■ pt/+page.svelte
■ ■■■ dl/+page.svelte
■■■ static/
■ ■■■ audio/ # all 5 mp3s
■ ■■■ images/
■ ■■■ docs/ # pdf + docx
■ ■■■ auth/hashes.json
■ ■■■ manifest.webmanifest # generated by vite-pwa
■ ■■■ favicon.svg
■■■ scripts/
■ ■■■ hash-passwords.mjs # build-time scrypt hash
■ ■■■ migrate.mjs # one-shot localStorage → Dexie
■■■ tests/
■ ■■■ unit/ # vitest
■ ■■■ e2e/ # playwright
■■■ .gitignore
■■■ package.json
■■■ svelte.config.js
■■■ vite.config.ts
■■■ tsconfig.json
■■■ netlify.toml # updated for npm build
■■■ README.md # full rewrite
■■■ PRESERVATION.md # checklist of all preserved items
■■■ docs/
■■■ architecture.md
■■■ adding-a-sub-app.md
■■■ deployment.md
```

Design system

- **Mobile-first**, breakpoints: `sm 640`, `md 768`, `lg 1024`, `xl 1280`, `2xl 1536`. Must look great at 360px width (smallest realistic phone) and 4K TV.
- **Typography scale:** 12 / 14 / 16 / 18 / 20 / 24 / 30 / 36 / 48 (Tailwind-like modular scale, base 16).
- **Spacing:** 4px grid (4, 8, 12, 16, 24, 32, 48, 64).
- **Colors:** primary `#1f2e4a` (current navy), accent pink `#ec4899`, success green `#10b981`, warning amber, error red. All as CSS custom properties so dark mode is a one-line `:root.dark` override.
- **Dark mode:** `prefers-color-scheme` default + manual toggle in settings. All components must support both from day one.
- **Accessibility:** keyboard nav, focus rings, ARIA labels, `prefers-reduced-motion` to disable confetti

and animations, color contrast ≥ WCAG AA.
- **Motion:** respect `prefers-reduced-motion`. Default subtle (200ms ease-out). Confetti + easter eggs gated behind the user explicitly enabling "fun mode" in settings.

Easter eggs in the new app

All current ones preserved. Plus add these subtle ones:
- Type `presuntinho` anywhere → unlocks "Mascot Master" badge
- Type `fatma` → unlocks "For Fatma" badge
- Type `love` → unlocks "Made with ❤■" badge
- Triple-click the hub logo (now bigger) → secret room with all easter egg lore
- Hold `?` for 3 seconds → keyboard shortcut overlay

Routing & data

- **Client-side routing only** (SPA mode via SvelteKit's `adapter-static`). No SSR — keeps Netlify config trivial, all data is local.
- **First-load:** splash → enter passwords → hub dashboard → user clicks sub-app → that sub-app's data hydrates from Dexie on demand (lazy load).
- **State shape preserved exactly** so existing user progress (XP, badges, quiz scores, secret discoveries, heartClicks, etc.) survives the migration. The migration script reads current localStorage, writes to Dexie, never deletes the localStorage key until user explicitly clears data in settings.

EXECUTION PLAN — SMALL, VERIFIABLE TASKS

EXECUTION PLAN — ORCHESTRATED 3-AGENT (Skander 1 / Skander 2 / Piccolo)

Orchestration model — READ FIRST

You (this agent, the principal) **DO NOT IMPLEMENT**. You **ORCHESTRATE**. You have three sub-agents available:

- **Skander 1** — senior reasoning agent. Use for: architecture decisions, code review, debugging, design trade-offs, refactor planning, accessibility audits, performance analysis. **Reasoning-heavy work.**
- **Skander 2** — mechanical implementation agent. Use for: extracting files, writing repetitive code from a spec, generating JSON datasets, running scripts, generating boilerplate, file moves, git operations, dep installs, builds, tests. **Mechanical work with clear spec.**
- **Piccolo** — fast execution agent. Use for: small one-shot tasks (single function write, single config edit, single file read+verify, single git push). **Sub-30-second tasks.**

You = orchestrator. Your job is to:
1. Read context, plan the phase
2. Decompose each phase into discrete tasks with clear deliverables
3. **Dispatch the right agent to each task** via `delegate_task`
4. **Verify the result** (browser-test, build-test, smoke-test) — never trust a sub-agent's self-report without verification
5. **Catch integration issues** between sub-agent outputs
6. **Coordinate git commits** — each phase ends with one commit + push to `main`
7. **Gate between phases** — do NOT proceed to phase N+1 until phase N has a green Netlify deploy

Sub-agent rules:
- Skander 1 / Skander 2 run via `delegate_task` (background, return when done)
- Each delegated task MUST be self-contained: include file paths, expected output, exact constraints
- Each delegated task MUST return a verifiable handle (file path, commit SHA, HTTP status, test output)
- **You verify every handle yourself** before marking the task done — sub-agent self-reports are NOT proof
- Piccolo is for tiny things you can do in 1-2 tool calls; do not over-use him for real work
- Maximum 3 sub-agents running in parallel (system limit). Do not exceed.
- Sub-agents cannot call you back. If a sub-agent gets stuck, dispatch a follow-up task with the additional context.

Anti-patterns to avoid:
- ■ Doing implementation work yourself when a sub-agent could do it faster
- ■ Dispatching huge "build me the whole V4" tasks — split into ≤90-min pieces
- ■ Trusting sub-agent claims without verifying the artifact
- ■ Skipping phase gates ("deploy first, then next phase")
- ■ Letting sub-agents make architecture decisions (those are yours / Skander 1's job)

Phase 0 — Recon (you, principal, do this directly)

No delegation. This is judgment work.

1. Read every file in current repo. List each file, its purpose, what to preserve.
2. Write `/PRESERVATION.md` with the 13-item inventory. Commit.
3. Test deployed site manually (browser_navigate https://presuntinho.netlify.app/, click every link, click heart 5×, try Konami, dump localStorage).
4. Check Node/npm versions (`node -v`, `npm -v`).

5. Check if `delegate_task` works by dispatching a trivial Piccolo test (e.g. "echo the SHA of HEAD").
6. **Gate:** do not proceed until you have the inventory committed and confirmed working.

Phase 1 — Bootstrap (split: Piccolo + Skander 2 + Skander 1 review)

7. **Piccolo** (≤2 min): create `package.json` with the dep list. Verify `npm install` succeeds.
8. **Skander 2** (≤15 min): scaffold SvelteKit project (`npx sv create .` non-interactive flags OR manual `svelte.config.js` + `vite.config.ts` + `tsconfig.json` + `src/app.html`). Add `dexie`, `@vite-pwa/sveltekit`, `chart.js`, `date-fns`, `lucide-svelte`, `svelte-i18n`, `vitest`, `@playwright/test`. Verify `npm run build` produces a `build/` directory.
9. **Skander 2** (≤10 min): move current HTML/JS/CSS into `/static/legacy/`. Set `netlify.toml` build = `npm run build`, publish = `build/`. Create minimal `src/routes/+page.svelte` that serves the legacy index.html inside an `<iframe>`.
10. **Skander 1** (≤10 min): **review the bootstrap** — verify the legacy iframe hack actually works (open `/` in browser, see current site), verify build pipeline, verify Netlify deploy config is correct. Report any blocking issues. **Do NOT skip this review.**
11. **You (principal)**: commit + push. Verify Netlify deploys. **Gate:** site must look identical to current at https://presuntinho.netlify.app/.

Phase 2 — Auth + layout shell (Skander 2 builds, Skander 1 reviews)

12. **Skander 2** (≤20 min): build `/splash` password gate. Web Crypto scrypt hash both passwords at build time via `scripts/hash-passwords.mjs`. Commit hashes to `/static/auth/hashes.json`. Splash UI: large mascot, password input, "Enter" button, ❤■ "Made for Fatma" footer. 3-strike lockout with localStorage timer. Wrong-password shake animation.
13. **Skander 2** (≤20 min): build `+layout.svelte` with top nav (logo + theme toggle + settings cog). Build `+layout.ts` to redirect to `/splash` if no valid session in sessionStorage. Build `+page.svelte` (hub) with placeholder cards for all 5 sub-apps from `src/lib/registry.ts`.
14. **Skander 1** (≤15 min): **security review** — verify the scrypt hash comparison is constant-time-ish, verify sessionStorage gating covers all routes, verify no password plaintext leaks in source. **Blocker if security issues.**
15. **You**: commit + push. Browser-test splash → hub flow on Netlify. **Gate:** both passwords work, both lock out at 3 strikes, hub renders empty cards.

Phase 3 — Migration + state (Skander 2 implements, Skander 1 reviews)

16. **Skander 2** (≤20 min): write Dexie schema in `src/lib/state/db.ts` mirroring all 11 current state keys (`xp`, `badges`, `visited`, `heartClicks`, `logoClicks`, `logoTimer`, `konamiProg`, `keyBuf`, `footerClicks`, `quizScore`, `quizAnswered`, `secretDiscovered`). Add indexes on `visited` and `secretDiscovered`.
17. **Skander 2** (≤15 min): write `src/lib/state/migration.ts` — on first run, read current `localStorage.presuntinho` key, parse, write to Dexie. Keep localStorage key for one rollback cycle. Log migration success/failure to console.
18. **Skander 2** (≤30 min): port `state.js` logic into Svelte stores (`src/lib/state/stores/{user,progress,easterEggs,settings}.ts`). Port `easter-eggs.js` into a single `src/lib/easterEggs/index.ts` module + a Svelte component `Confetti.svelte` + a `<SecretModal>` component. Port `quizzes.js` into a data-driven `QuizRunner.svelte` that reads from `/static/quizzes/*.json`.
19. **Skander 1** (≤20 min): **data-migration review** — open the deployed site, do all easter eggs, refresh page, verify XP/badges/clicks survived. Specifically test: heart 5×, logo 3×, type "perfume", complete one quiz, refresh, check `db.xp.toArray()` in DevTools.
20. **You**: commit + push. **Gate:** every current state key survives refresh via Dexie.

Phase 4 — Escola sub-app (Skander 2 builds lessons, Skander 1 reviews pedagogy)

21. **Skander 2** (≤30 min): read current `index.html` lines covering SWOT/Persona/Problem/TOWS/Recommendation. Extract into 5 structured lesson JSONs in `/static/lessons/equivalenza/{swot,persona,problem,tows,recommendation}.json`. Each has: id, title, audio path, sections array (h2/p/ul/callout), quizSlug, keyPoints.
22. **Skander 2** (≤30 min): extract 5 quizzes into `/static/quizzes/{q1,q2,q3,q4,pt}.json` with proper schema (id, courseId, questions[{prompt, choices, correctIndex, explanation}]).
23. **Skander 2** (≤45 min): build `LessonRunner.svelte` (renders sections from JSON, plays audio, shows key points, next/prev nav, progress dots). Build `QuizRunner.svelte` (one question at a time, immediate feedback, score persistence). Wire routes: `/escola`, `/escola/curso/[slug]`, `/escola/licao/[courseSlug]/[lessonSlug]`, `/escola/quiz/[quizSlug]`.
24. **Skander 1** (≤15 min): **content review** — verify the lesson JSONs preserve all original educational content (no paragraphs dropped, all key insights intact, audio references correct, quiz explanations make pedagogical sense).
25. **You**: commit + push. Browser-test every lesson URL + every quiz URL on Netlify. **Gate:** all 5 lessons + all 5 quizzes reachable, audio plays, quiz answers persist.

Phase 5 — Trabalhos (Skander 2, fast)

26. **Skander 2** (≤20 min): build `/trabalhos` list page + `/trabalhos/assignment/equivalenza` hub. Move PDFs, DOCX, ZIP from legacy `/dl` to this hub. Add deadline countdown widget.
27. **You**: commit + push. **Gate:** all 3 documents downloadable from the new route.

Phase 6 — Finanças MVP (Skander 2 builds, Skander 1 reviews)

28. **Skander 2** (≤30 min): Dexie tables `transacoes`, `orcamentos`, `categorias`. Seed 10 categories (Alimentação, Transporte, etc) on first run. Build `/financas` dashboard with totals + a single bar chart (chart.js, monthly spending).
29. **Skander 2** (≤45 min): `/financas/transacoes` (list with month/category filters), `/financas/orcamento` (set per-category monthly limits, show progress bar), `/financas/nova` (add-transaction form with category dropdown, type toggle, amount input, date picker).
30. **Skander 1** (≤15 min): **UX review** — test the add-transaction flow, verify data persists across reloads, verify chart re-renders on data change, verify decimal/currency formatting works for pt-PT (comma as decimal sep).
31. **You**: commit + push. **Gate:** can add a transaction, see it on dashboard, persists after refresh.

Phase 7 — Hábitos MVP (Skander 2, Skander 1 reviews)

32. **Skander 2** (≤30 min): Dexie tables `habitos` (id, name, icon, color, cadence) + `habit_logs` (id, habitId, date, done). Build `/habitos` list with streak counter (current streak = consecutive days with log=true).
33. **Skander 2** (≤30 min): `/habitos/habit/[slug]` detail with SVG calendar heatmap (last 90 days), log-today button. `/habitos/novo` add-habit form.
34. **Skander 1** (≤10 min): verify heatmap SVG renders correctly, streak logic handles DST/timezone, missing days don't break streak.
35. **You**: commit + push. **Gate:** create habit, log it, refresh, streak still shows.

Phase 8 — Biblioteca MVP (Skander 2, quick)

36. **Skander 2** (≤30 min): Dexie `biblioteca` table (id, title, url, tags[], description, createdAt). Build `/biblioteca` list (search by title/tag), `/biblioteca/novo` form, `/biblioteca/item/[id]` detail.
37. **You**: commit + push. **Gate:** add bookmark, search, refresh, still there.

Phase 9 — Settings + i18n (Skander 2 builds, Skander 1 reviews)

38. **Skander 2** (≤30 min): `/definicoes` settings page — theme toggle (light/dark/auto, persisted to Dexie `settings.theme`), language toggle (pt-PT/en, persisted), reset-password (re-prompts for current password), clear-data (modal confirm, clears all Dexie tables except auth), export-JSON, import-JSON.
39. **Skander 2** (≤30 min): wire `svelte-i18n`. Externalize all UI strings to `/src/lib/i18n/{pt-PT,en}.json`. Replace hard-coded strings in every component. Localize `/splash` too. Default locale: pt-PT (read from `localStorage.fat-pref-lang`, fallback to pt-PT).
40. **Skander 1** (≤15 min): **i18n review** — grep for hard-coded English strings still in components, verify pt-PT copy is natural Portuguese (not Brazilian), verify all UI surfaces have translations.
41. **You**: commit + push. Toggle language on Netlify, verify all labels change. **Gate:** zero hard-coded user-facing strings remain.

Phase 10 — Polish + a11y (Skander 1 leads)

42. **Skander 1** (≤30 min): **full a11y audit** — every interactive element has ARIA label or accessible name, every form has labels not placeholders, focus rings visible on all focusable elements, `tabindex` correct, skip-to-content link present, heading hierarchy correct (no h1→h3 skips), color contrast ≥ 4.5:1 for normal text, ≥ 3:1 for large.
43. **Skander 1** (≤20 min): **motion audit** — every animation wrapped in `if (!window.matchMedia('(prefers-reduced-motion: reduce)').matches)`. Confetti, mascot bounce, card hover effects all gated.
44. **Skander 2** (≤20 min): **Lighthouse** — run `npx lighthouse https://presuntinho.netlify.app/ --preset=mobile --output=json --output-path=./lh.json`. Target ≥ 95 on perf, a11y, best-practices, SEO. Fix any < 90 score blocker.
45. **Skander 2** (≤20 min): **responsive test** — Chrome DevTools device emulation at 360×640, 768×1024, 1280×800, 1920×1080, 3840×2160. Fix any layout break.
46. **Skander 1** (≤15 min): **final code review** — TypeScript strict mode clean, no `any` without justification, no console warnings, bundle size < 150 KB gz initial.
47. **You**: rewrite `/README.md` with v4 architecture, how to add a sub-app, deploy instructions. Commit.

Phase 11 — Final cleanup + tag

48. **Skander 2** (≤10 min): delete `/static/legacy/` only after Skander 1 confirms every legacy URL has a redirect in `+page.ts` or has been replaced.
49. **You**: tag `v4.0.0`. `git tag -a v4.0.0 -m "V4: SvelteKit PWA hub with escola/trabalhos/finanças/hábitos/biblioteca sub-apps"`. Push tag. Verify Netlify deploys final build.
50. **You**: post final summary to user with deploy URL, live sub-apps list, preserved features verified, test results, deferred v5 items.

Orchestration cadence (how to actually run this)

Each phase takes ~2 hours wall-clock with 3-agent orchestration. Total: ~22 hours of agent work but with parallel delegation you'll finish in **3–5 days** of real session time.

Your daily rhythm:
1. Start of session: `git pull`, run `git log --oneline -5`, check `PRESERVATION.md` for current state.
2. Pick the next phase. Read its steps.
3. Dispatch Phase N's tasks. While waiting, do your own recon for Phase N+1.
4. As sub-agents return, verify each artifact. Catch integration bugs early.
5. Commit + push at end of each phase.
6. End of session: write a status note (what's done, what's next, blockers) into `/STATUS.md` so the next session's principal can resume without re-reading everything.

When to NOT delegate and do it yourself:
- Reading context (always you)
- Architecture decisions (always you, possibly with Skander 1 input)
- Final verification of any sub-agent output (always you)
- Writing the principal-level `PRESERVATION.md` / `STATUS.md` (always you)

When to use Skander 1:
- "What's the right way to split this content into lessons?" — design question
- "Review this code for security issues" — review
- "Review this i18n for natural pt-PT" — language/cultural review
- "Why is Lighthouse perf 78?" — debugging
- "Audit accessibility of this whole page" — full audit

When to use Skander 2:
- "Generate these 5 JSON lesson files from this HTML" — mechanical extraction
- "Wire Dexie tables for these 3 entities" — boilerplate
- "Run npm install and verify build succeeds" — scripted task
- "Move all these files into /static/legacy/" — mechanical

When to use Piccolo:
- "Echo the current SHA of HEAD" — 1-line query
- "Update the version string in package.json" — single edit
- "Confirm this file exists at that path" — single check

Maximum parallelism rule: never run more than 3 sub-agents at once. If a phase has 5 tasks, batch them: 3 in parallel, then the remaining 2 when one returns.

QUALITY BAR

- **TypeScript strict mode.** No `any` unless absolutely necessary.
- **Every component tested** with vitest unit + at least one Playwright e2e.
- **No console errors** in production build.
- **No console warnings** (Svelte a11y warnings must be 0).
- **Lighthouse mobile score ≥ 95** on home + escola hub + finanças dashboard.
- **Bundle size**: initial route < 150 KB gz, total app < 500 KB gz.
- **Works offline** after first visit (PWA service worker).
- **Installable** to home screen on iOS Safari + Android Chrome.
- **All current content reachable**: every existing page has a corresponding new URL.
- **All current easter eggs still triggerable**: heart, logo, konami, keywords, footer, secret room.
- **All current state preserved**: XP, badges, quiz scores, heartClicks etc. survive migration.
- **No emoji character entity references** like `&#x1F3E0;` — use real emoji in source.

ABSOLUTELY DO NOT

- ■ Do not delete or rewrite any file without first preserving it in `/static/legacy/` or in the new repo.
- ■ Do not change Netlify's auto-deploy URL — `presuntinho.netlify.app` must keep working throughout the migration.
- ■ Do not introduce a backend or database service. V4 is fully client-side.
- ■ Do not skip the verification step at the end of each phase. **Each phase ends with a successful Netlify deploy of that phase's work.**
- ■ Do not commit `.env`, `node_modules`, `build/`, `.svelte-kit/`, or any secrets.
- ■ Do not use placeholder lorem ipsum in production UI strings. Real pt-PT copy throughout.
- ■ Do not merge unrelated PRs / work into one commit. One logical change per commit.
- ■ Do not remove the `■■ PT` tab or the Portuguese quiz. Fatma's primary language is Portuguese.

DELIVERABLES

When you're done:
1. Push to `main` branch. Netlify auto-deploys.
2. Verify https://presuntinho.netlify.app/ works end-to-end manually.
3. Run `npm run build` — must exit 0.
4. Run `npm run test` — must exit 0 (all tests pass).
5. Run `npx playwright test` — must exit 0 (e2e flows pass).
6. Update `/PRESERVATION.md` checking off every preserved item.
7. Write `/docs/architecture.md` explaining the sub-app registry pattern.

8. Write `/docs/adding-a-sub-app.md` — a developer guide showing how to add a 6th sub-app in <30 min.
9. Write `/STATUS.md` for the next session's principal — what's done, what's next, any blockers.
10. Print a final summary message to the user with: deploy URL, list of sub-apps live, list of preserved features verified, test results, and any deferred items for v5.

FINAL WORD

This is a real, personal app for a real person (Fatma). Treat her data with care, her language with respect, and her time with urgency. The current site is already deployed and live — your job is to make it **structurally better** without making it **functionally worse** at any step. Commit often, deploy often, verify after every phase. If you hit a blocker you cannot resolve, stop and ask the user — do not invent workarounds that look correct but aren't.

Begin.